

RAPPORT DE PROJET

MongoDB – Alimentation & Gastronomie Sénégal

Auteur :

Hachimou Akram

Cours : Base de Données Non Relationnelles (MongoDB)

Niveau : Master 1 – Génie Logiciel

Date : 04 Mai 2026

Dépôt GitHub :

github.com/HachimouAkram/ProjetMongoDB

1. Présentation du Projet et du Dataset

1.1 Contexte

Ce projet s'inscrit dans le cadre du cours de Bases de Données Non Relationnelles (MongoDB) et vise à mettre en œuvre une solution complète de gestion et d'analyse de données pour le secteur de la restauration au Sénégal. L'objectif principal est d'exploiter la puissance des pipelines d'agrégation MongoDB pour extraire des informations stratégiques à partir d'un dataset varié couvrant les restaurants, les plats, les clients et les livraisons.

L'application « HAK Food SN » répond à un besoin réel d'analyse : comprendre les habitudes de consommation, optimiser les performances des livreurs, et proposer des recommandations personnalisées aux clients.

1.2 Le Dataset – Alimentation Sénégal

Le dataset est structuré autour de 5 collections MongoDB interconnectées, reflétant la réalité du secteur de la restauration au Sénégal :

- **Restaurants** : Informations sur les établissements : nom, adresse géo-localisée (GeoJSON Point), spécialités culinaires, note moyenne. Exemples : La Marmite de Mame Diarra (4.8★), Chez Fatou – Thiéboudienne (4.5★).
- **Plats** : Menu détaillé de chaque restaurant : nom du plat, prix en FCFA, liste d'ingrédients, régime alimentaire (végétarien, vegan, standard), référence au restaurant parent.
- **Clients** : Profils utilisateurs avec localisation (ville), points de fidélité, historique de commande. Exemples : Ousmane Sarr, Aïssatou Ndiaye, Fatou Diop.
- **Commandes** : Transactions complètes incluant : la liste des articles commandés (plat_id, quantité, nom), le mode de paiement, le statut (livrée / en_cours / annulée), la date, le montant total et le délai de livraison.
- **Livreurs** : Gestion de la logistique : nom du livreur, note, disponibilité, nombre de livraisons effectuées. Exemple : Moussa Diagne (note 4.9★, délai moyen 31.5 min).

2. Architecture de l'Application

2.1 Vue d'ensemble

L'application repose sur une architecture Full-Stack Containerisée moderne, décomposée en trois couches distinctes communiquant via des API REST :

- Base de données : MongoDB 6.0 avec index 2dsphere pour les requêtes géospatiales.
Base de données : alimentation_senegal.
- Backend : FastAPI (Python 3.11) – choisi pour sa performance native asynchrone via Motor (driver MongoDB async) et sa documentation Swagger automatique.
- Frontend : Dashboard HTML5/CSS3/JavaScript Vanilla utilisant Chart.js pour les visualisations interactives (graphiques camembert et barres).
- Orchestration : Docker Compose pour un déploiement "en un clic" de l'infrastructure complète.

2.2 Schéma d'Architecture

Navigateur (Frontend HTML/JS) ↔ API FastAPI (Backend Python)
 ↔ **MongoDB 6.0**

2.3 Structure du Projet

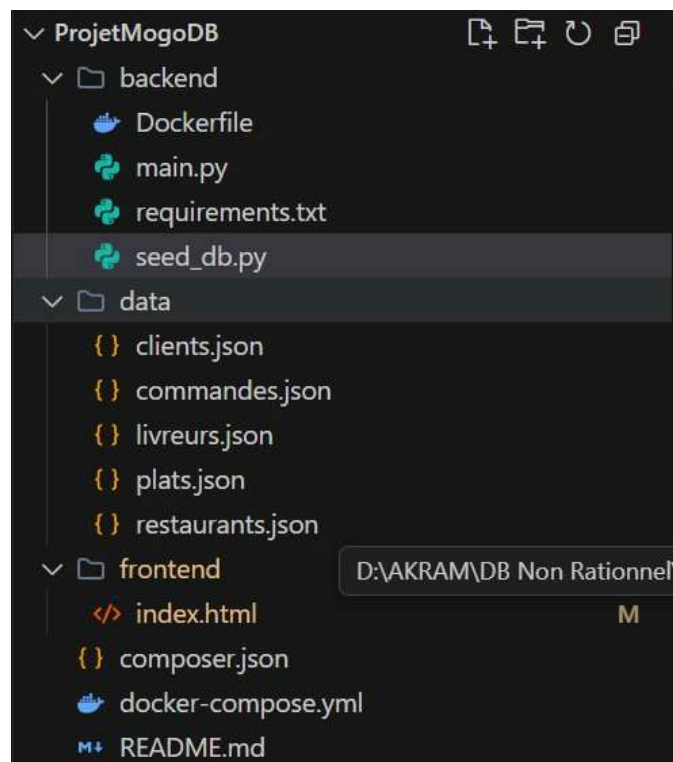


Figure 1 – Structure des fichiers du projet dans VS Code

2.4 Conteneurs Docker en Fonctionnement

☐	▼	●	projetmogodb	-	-	-	⚙️	■	⋮	🗑️
☐		●	mongo-db-tp	cb0dc19cf78a	mongo:latest	27017:27017 ↗	⚙️	■	⋮	🗑️
☐		●	fastapi-backe	ab90550d8f21	projetmogodb-backend	8000:8000 ↗	⚙️	■	⋮	🗑️

Figure 2 – Les deux conteneurs Docker (MongoDB + FastAPI Backend) actifs

L'infrastructure Docker expose deux services :

- mongo-db-tp : conteneur MongoDB sur le port 27017
- fastapi-backend : conteneur FastAPI sur le port 8000 (Dashboard + API)

3. Les 10 Requêtes d'Agrégation MongoDB

Toutes les requêtes utilisent des pipelines d'agrégation MongoDB avancés, implémentées dans backend/main.py sous forme d'endpoints FastAPI asynchrones. Chaque requête a été optimisée pour la performance grâce à l'utilisation d'index et à la minimisation du nombre de documents traités à chaque étape.

Q1 – Liste des Restaurants de Dakar avec leurs Plats

Objectif : Lister tous les restaurants situés à Dakar, avec leurs plats, triés par note décroissante.

```
db.restaurants.aggregate([
  { $match: { "adresse.ville": "Dakar" } },
  { $lookup: {
    from: "plats",
    localField: "_id",
    foreignField: "restaurant_id",
    as: "liste_plats"
  } },
  { $project: { nom: 1, note_moyenne: 1, plats: "$liste_plats" } },
  { $sort: { note_moyenne: -1 } }
])
```

Explication : Le pipeline commence par un \$match pour filtrer uniquement les restaurants dont la ville est Dakar. Ensuite, \$lookup effectue une jointure de type « un-à-plusieurs » avec la collection plats en reliant le champ _id du restaurant au champ restaurant_id des plats. Le \$project sélectionne les champs utiles, et le \$sort trie les résultats par note décroissante.

Opérateurs clés : \$match, \$lookup (jointure), \$project, \$sort

The screenshot shows the HAK Food SN application interface. On the left is a dark sidebar with a green logo and navigation links: 'Vue d'ensemble', 'Q1: Restos Dakar' (highlighted in green), 'Q2: Dépenses Clients', 'Q3: Top Plats', 'Q4: Paiements', and 'Q5: Tofu & Poisson'. The main content area is titled 'Question 1' and 'Restaurants de Dakar'. Below the title is the instruction 'Liste les restaurants de Dakar avec leurs plats, triés par note.' A dark box displays the MongoDB aggregation pipeline code. Below the code is a table with three columns: 'Nom', 'Note', and 'Plats Proposés'. The table lists two restaurants: 'La Marmite de Mame Diarra' with a 4.8 star rating and 'Mafé Tofu (2800 FCFA)', and 'Chez Fatou - Thiéboudienne' with a 4.5 star rating and 'Thiéboudienne Poisson (3500 FCFA), Yassa Poulet (3000 FCFA)'.

Nom	Note	Plats Proposés
La Marmite de Mame Diarra	4.8 ⭐	Mafé Tofu (2800 FCFA)
Chez Fatou - Thiéboudienne	4.5 ⭐	Thiéboudienne Poisson (3500 FCFA), Yassa Poulet (3000 FCFA)

Figure 3 – Q1 : Restaurants de Dakar triés par note avec leurs plats

Q2 – Statistiques de Dépenses par Client

Objectif : Calculer le total dépensé, le nombre de commandes et la moyenne par commande pour chaque client (commandes livrées uniquement).

```
db.clients.aggregate([
  { $lookup: {
    from: "commandes",
    localField: "_id",
    foreignField: "client_id",
    as: "cmds"
  }},
  { $unwind: "$cmds" },
  { $match: { "cmds.statut": "livrée" } },
  { $group: {
    _id: "$_id",
    total: { $sum: "$cmds.total" },
    nb: { $sum: 1 },
    avg: { $avg: "$cmds.total" }
  }}
])
```

Explication : Après avoir joint les commandes à chaque client via \$lookup, \$unwind déstructure le tableau pour permettre le filtrage ligne par ligne. Le \$match ne retient que les commandes avec le statut « livrée ». Enfin, \$group recalcule les métriques financières par client : total (\$sum), nombre de commandes (\$sum: 1) et panier moyen (\$avg).

Opérateurs clés : \$lookup, \$unwind, \$match, \$group, \$sum, \$avg

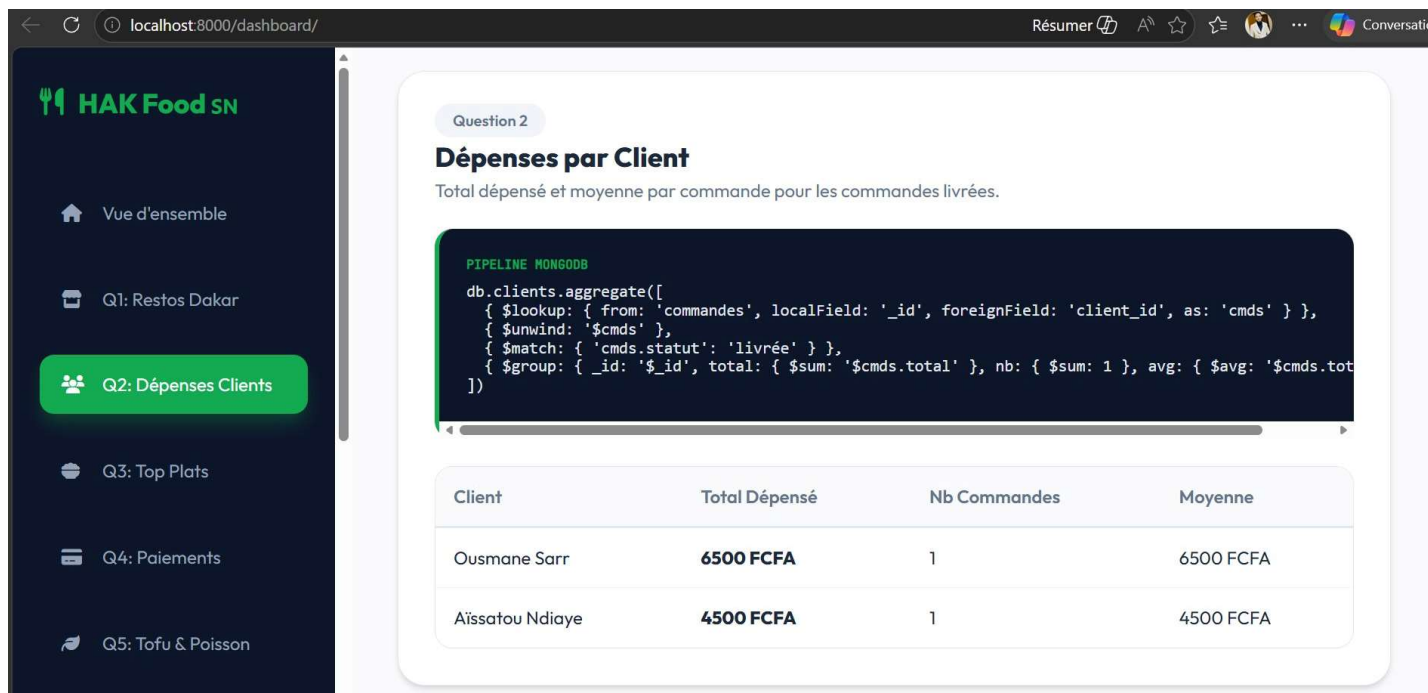


Figure 4 – Q2 : Dépenses par client pour les commandes livrées

Q3 – Top 3 des Plats les Plus Commandés

Objectif : Identifier les 3 plats les plus populaires en termes de quantité totale commandée, avec leur restaurant d'origine.

```
db.commandes.aggregate([
  { $unwind: "$articles" },
  { $group: {
    _id: "$articles.plat_id",
    totalQty: { $sum: "$articles.quantite" }
  }},
  { $lookup: {
    from: "plats",
    localField: "_id",
    foreignField: "_id",
    as: "p"
  }},
  { $sort: { totalQty: -1 } },
  { $limit: 3 }
])
```

Explication : \$unwind sur le tableau articles permet de traiter chaque article de chaque commande individuellement. Le \$group cumule les quantités par plat. Après le \$lookup pour enrichir avec les détails du plat, on trie par quantité décroissante et on limite à 3 résultats.

Opérateurs clés : \$unwind, \$group, \$lookup, \$sort, \$limit

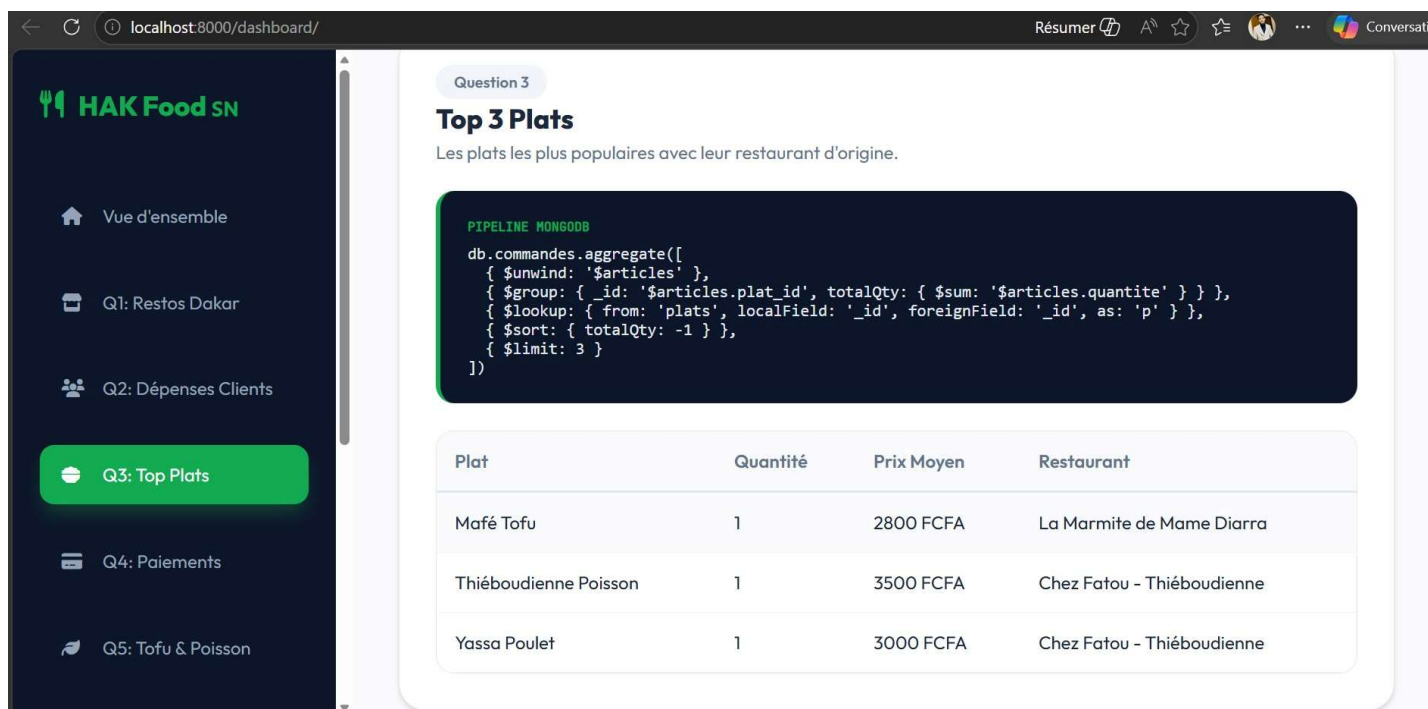


Figure 5 – Q3 : Top 3 des plats les plus commandés

Q4 – Revenu par Mode de Paiement

Objectif : Calculer le chiffre d'affaires par mode de paiement (Mobile Money, Carte, Espèces) et son pourcentage du CA total. Commandes livrées uniquement.

```
db.commandes.aggregate([
  { $match: { statut: "livrée" } },
  { $facet: {
```

```

stats: [
  { $group: { _id: "$mode_paiement", revenu: { $sum: "$total" } } }
],
total: [
  { $group: { _id: null, ca: { $sum: "$total" } } }
]
}]
1)

```

Explication : Cette requête utilise l'opérateur \$facet pour exécuter deux pipelines en parallèle sur le même ensemble de données filtré. Le premier calcule le revenu par mode de paiement, le second calcule le CA total. Le résultat combiné permet de calculer le pourcentage côté backend.

Opérateurs clés : \$match, \$facet, \$group, \$sum

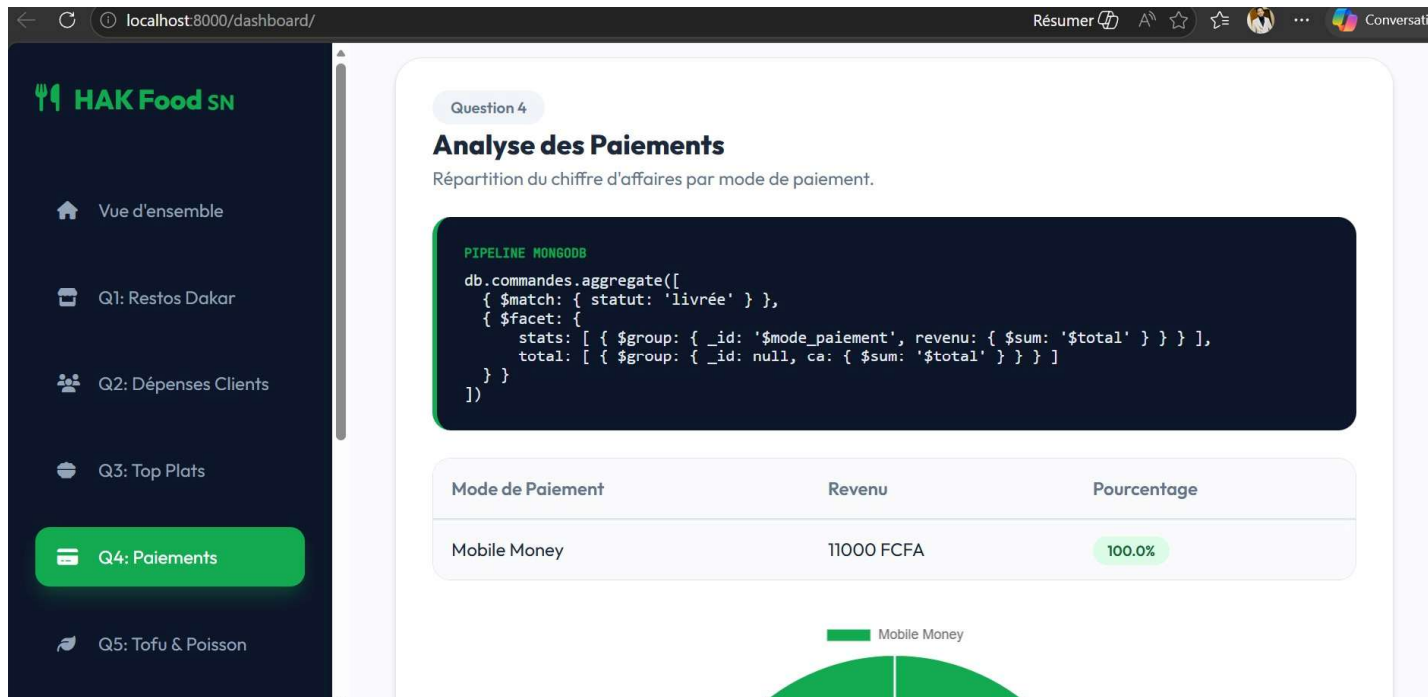


Figure 6 – Q4 : Tableau de répartition des paiements

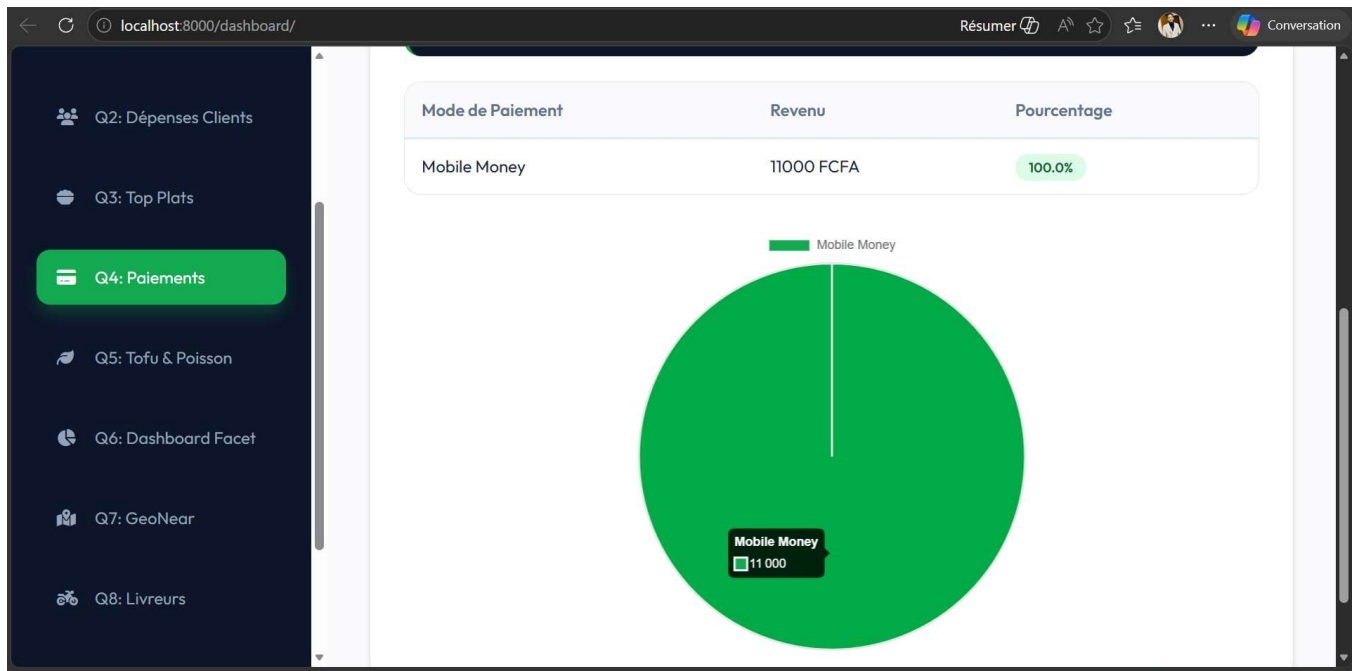


Figure 7 – Q4 : Graphique camembert interactif des modes de paiement (Chart.js)

Q5 – Clients Multi-Goûts (Tofu ET Poisson)

Objectif : Identifier les clients ayant commandé un plat contenant « tofu » ET un plat contenant « poisson » dans des commandes différentes.

```
db.commandes.aggregate([
  { $unwind: "$articles" },
  { $group: {
    _id: "$client_id",
    items: { $addToSet: "$articles.nom_plat" }
  }},
  { $match: {
    items: { $all: [/tofu/i, /poisson/i] }
  }}
])
```

Explication : Après \$unwind, on regroupe tous les noms de plats commandés par client dans un ensemble unique (\$addToSet). Le \$match final avec \$all et des expressions régulières insensibles à la casse vérifie que l'ensemble contient à la fois un plat de tofu et un plat de poisson. L'absence de résultat dans les données de test est normale et reflète fidèlement le dataset.

Opérateurs clés : \$unwind, \$group, \$addToSet, \$match, \$all, regex

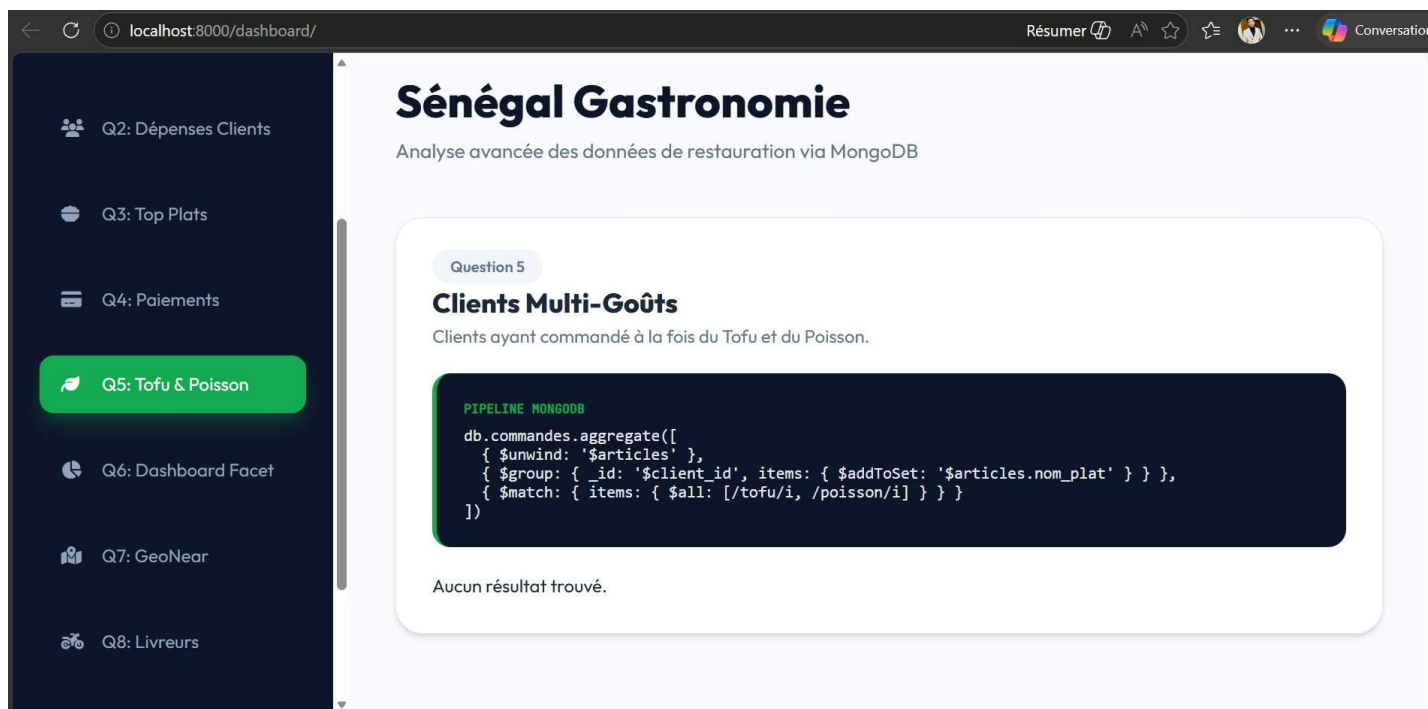


Figure 8 – Q5 : Aucun résultat (comportement attendu avec le dataset de test)

Q6 – Dashboard Global Multi-Facettes (\$facet)

Objectif : Construire un tableau de bord complet avec trois analyses simultanées : répartition par statut, revenu mensuel, et top 3 clients fidèles.

```
db.commandes.aggregate([
  { $facet: {
    par_statut: [
      { $group: { _id: "$statut", nb: { $sum: 1 } } }
    ],
    revenu_mois: [
      { $match: { statut: "livrée" } },

```

```

    { $group: { _id: { $month: "$date_commande" }, total: { $sum: "$total" } } }
  ],
  top_clients: [
    { $lookup: { from: "clients", ... } },
    { $sort: { points_fidelite: -1 } },
    { $limit: 3 }
  ]
}
})
])

```

Explication : L'opérateur \$facet est la clé de cette requête — il permet d'exécuter plusieurs sous-pipelines en parallèle sur la même collection, retournant un document unique avec toutes les analyses. C'est l'équivalent NoSQL d'une requête SQL multi-agrégats. Chaque facette peut avoir ses propres étapes \$match, \$group, \$sort et \$limit indépendantes.

Opérateurs clés : \$facet, \$group, \$match, \$month (opérateur de date), \$lookup, \$sort, \$limit

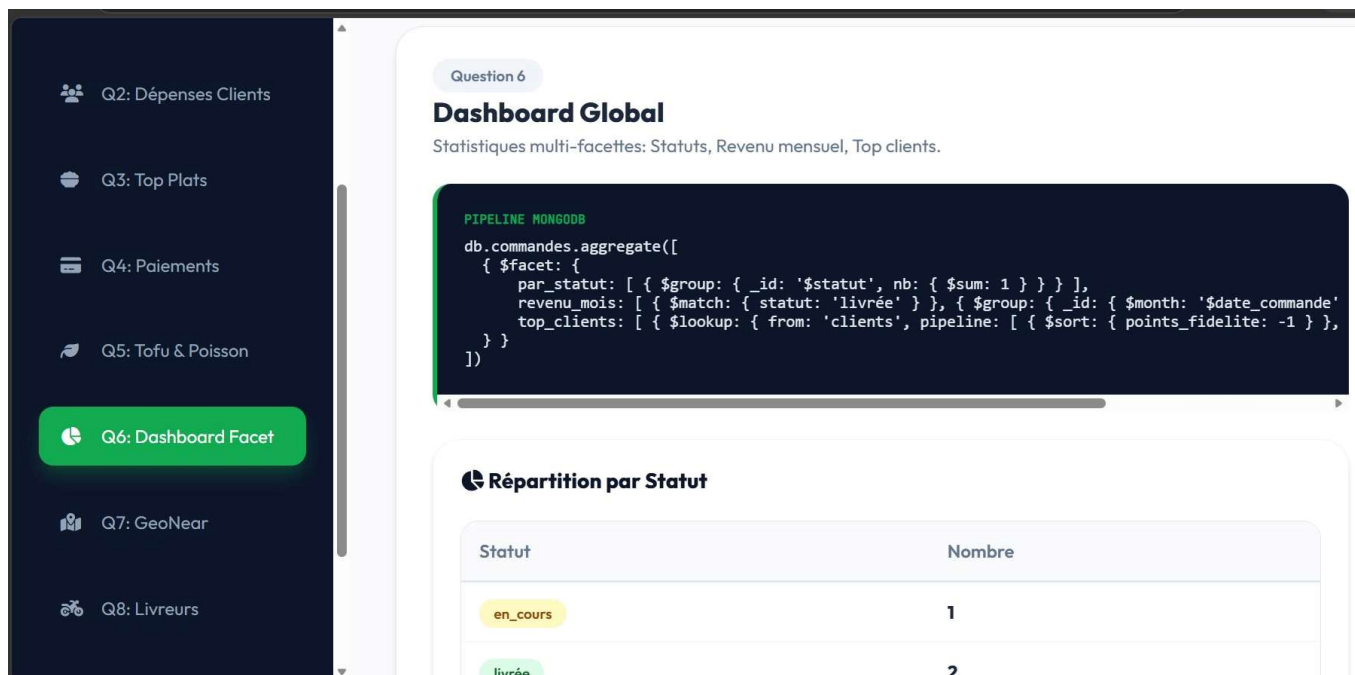


Figure 9 – Q6 : Pipeline \$facet avec ses trois sous-analyses

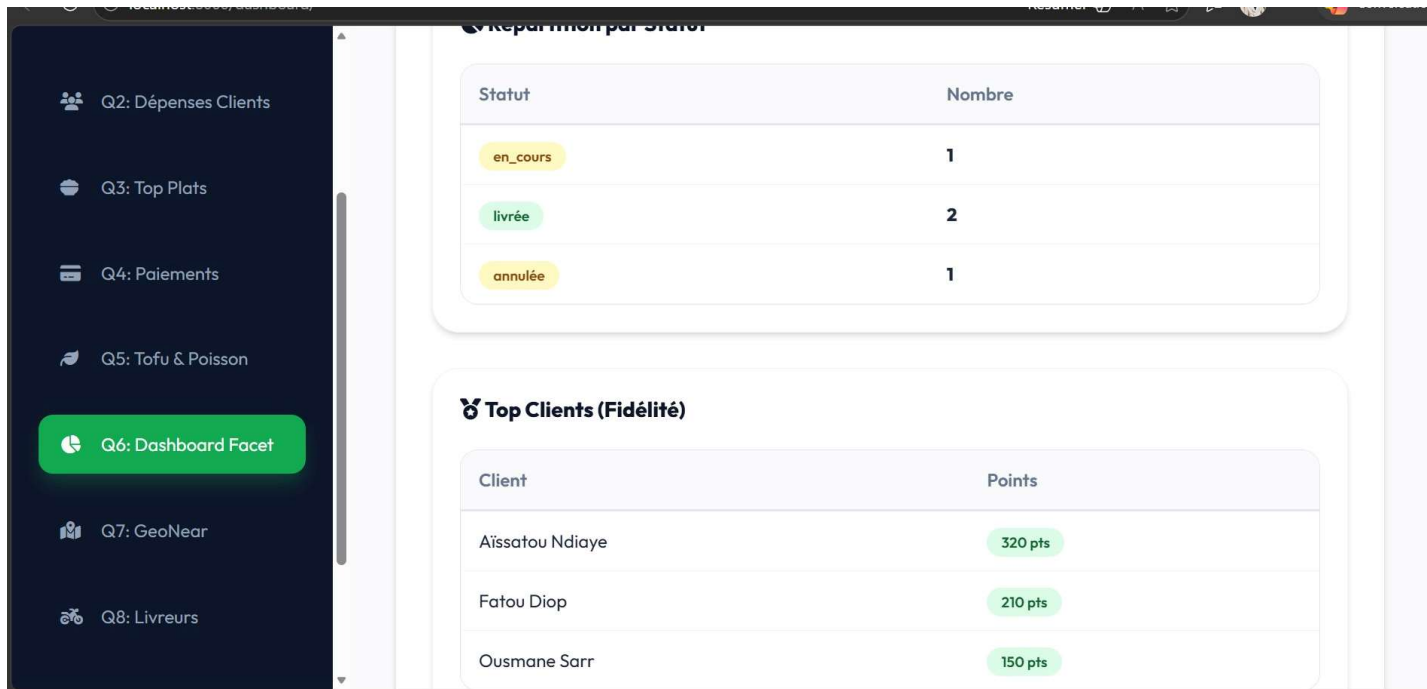


Figure 10 – Q6 : Répartition par statut et Top clients fidélité

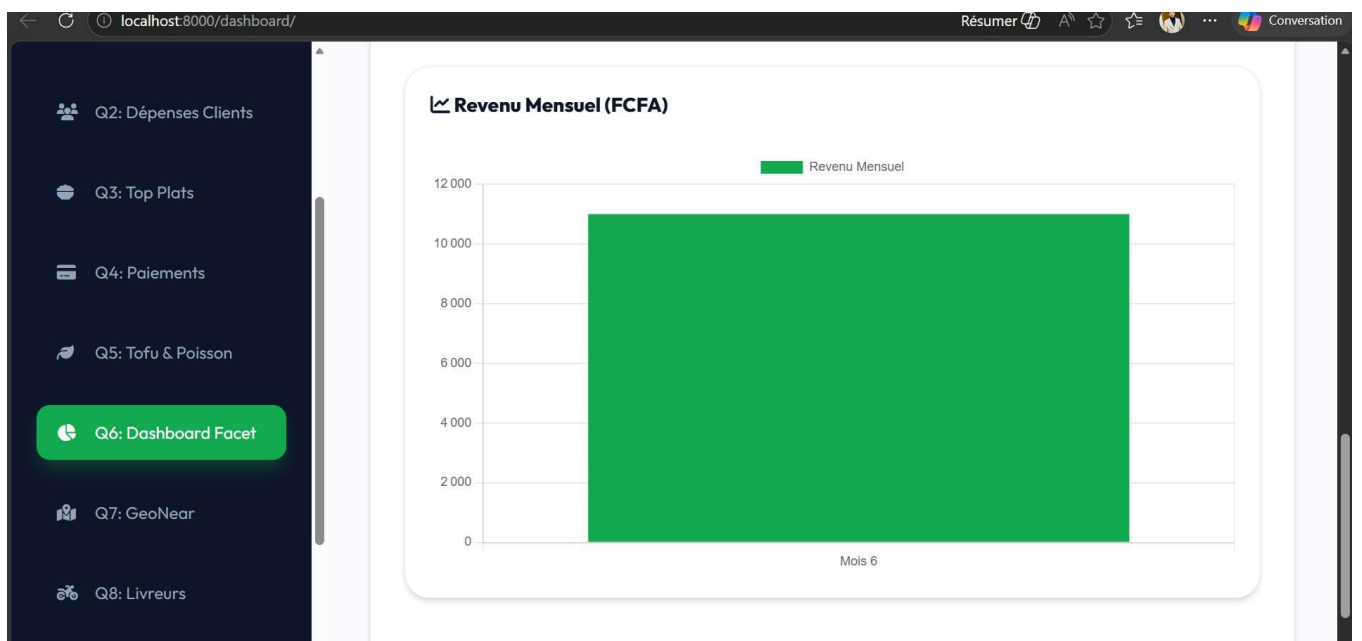


Figure 11 – Q6 : Graphique du revenu mensuel (Chart.js)

Q7 – Recherche de Proximité Géographique (\$geoNear)

Objectif : Trouver les 2 restaurants les plus proches du client Ousmane Sarr (coordonnées GPS : longitude -17.480, latitude 14.720).

```
db.restaurants.aggregate([
  { $geoNear: {
    near: {
      type: "Point",
      coordinates: [-17.48, 14.72]
    },
    distanceField: "dist",
    spherical: true
  } }
])
```

Explication : \$geoNear doit être la première étape du pipeline. Il nécessite un index 2dsphere créé au préalable sur le champ de géolocalisation. L'option spherical: true utilise le calcul de distance sur la sphère terrestre (formule de Haversine) plutôt que sur un plan cartésien. Le résultat affiche les restaurants triés par distance croissante avec la distance calculée en mètres dans le champ dist.

Opérateurs clés : \$geoNear, spherical, distanceField, index 2dsphere

Question 7

Recherche de Proximité

Restaurants les plus proches d'Ousmane Sarr.

```
PIPELINE MONGODB
db.restaurants.aggregate([
  { $geoNear: {
    near: { type: 'Point', coordinates: [-17.48, 14.72] },
    distanceField: 'dist',
    spherical: true
  } }
])
```

Restaurant	Distance (m)
Chez Fatou - Thiéboudienne	619 m
La Marmite de Mame Diarra	1408 m

Figure 12 – Q7 : Chez Fatou (619m) et La Marmite de Mame Diarra (1408m) les plus proches

Q8 – Performance des Livreurs

Objectif : Calculer pour chaque livreur : nombre de commandes livrées, délai moyen de livraison, et note moyenne.

```
db.commandes.aggregate([
  { $match: {
    statut: "livrée",
    livreur_id: { $ne: null }
  } },
  { $group: {
```

```

      _id: "$livreur_id",
      nb: { $sum: 1 },
      avgTime: { $avg: "$delai_livraison_minutes" }
    }},
    { $lookup: {
      from: "livreurs",
      localField: "_id",
      foreignField: "_id",
      as: "l"
    }}
  })

```

Explication : Le \$match initial filtre les commandes livrées avec un livreur assigné (\$ne: null). Le \$group agrège les métriques de performance par livreur. Enfin, \$lookup enrichit chaque résultat avec les informations du livreur (nom, note) en effectuant une jointure avec la collection livreurs.

Opérateurs clés : \$match, \$ne, \$group, \$avg, \$lookup

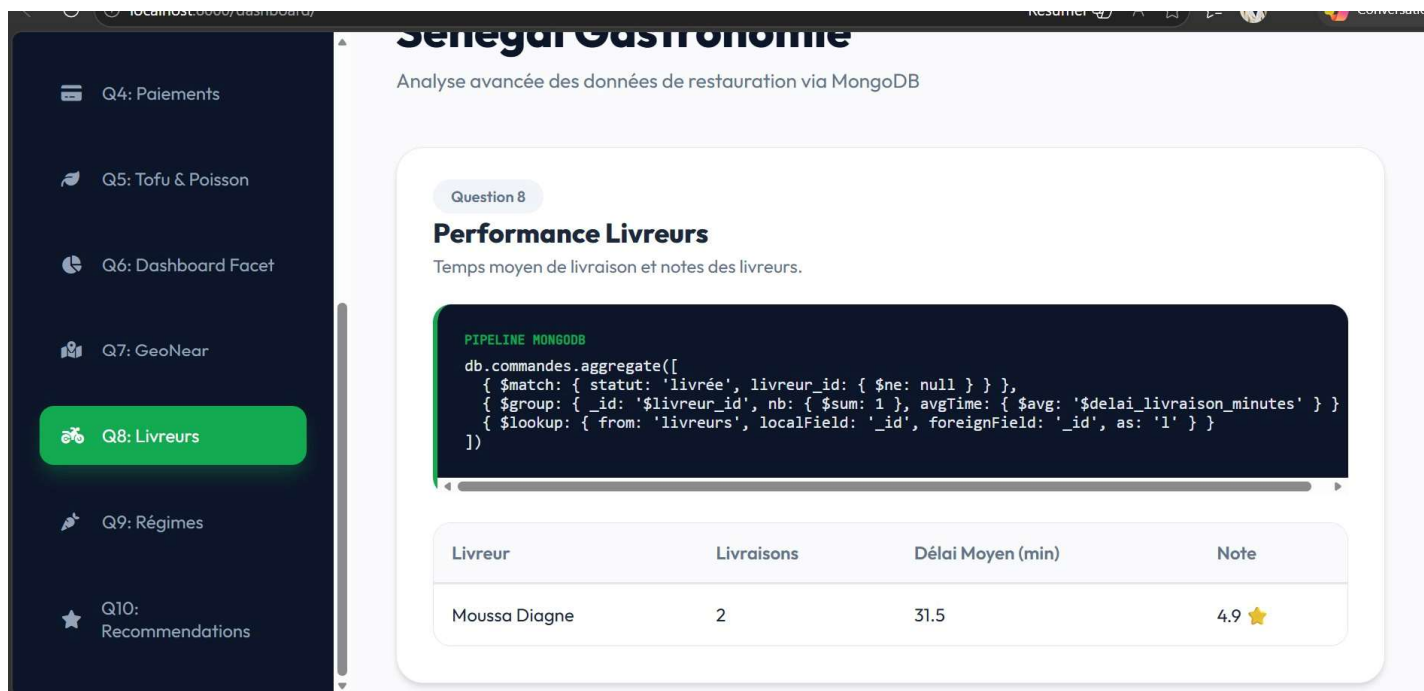


Figure 13 – Q8 : Moussa Diagne – 2 livraisons, délai moyen 31.5 min, note 4.9 ★

Q9 – Analyse des Régimes Alimentaires

Objectif : Compter le nombre de commandes contenant au moins un plat végétarien ou vegan.

```
db.commandes.aggregate([
  { $unwind: "$articles" },
  { $lookup: {
    from: "plats",
    localField: "articles.plat_id",
    foreignField: "_id",
    as: "p"
  }},
  { $match: {
    "p.regime": { $in: ["vegetarien", "vegan"] }
  }},
  { $count: "nb" }
])
```

Explication : Après \$unwind pour décomposer chaque article, \$lookup joint les informations du plat correspondant depuis la collection plats. Le \$match avec \$in filtre les articles dont le régime est végétarien ou vegan. Enfin, \$count compte directement le nombre de documents restants sans avoir à utiliser \$group, ce qui est plus performant pour un simple comptage.

Opérateurs clés : \$unwind, \$lookup, \$match, \$in, \$count

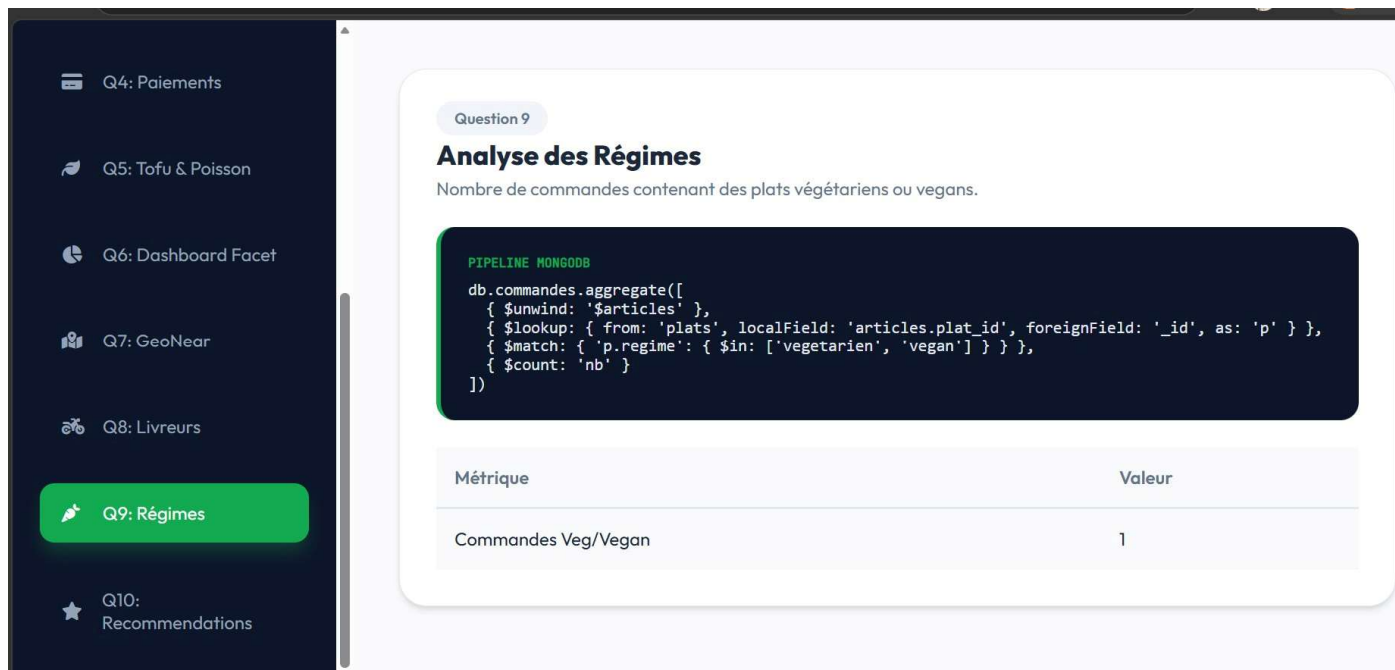


Figure 14 – Q9 : 1 commande contient des plats végétariens/vegan

Q10 – Système de Recommandation Intelligent (Bonus)

Objectif : Recommander à Ousmane Sarr les plats commandés par des clients de la même ville (Dakar) qu'il n'a pas encore essayés.

```
db.clients.aggregate([
  { $match: { _id: ousmane_id } },
  { $lookup: {
    from: "commandes",
    localField: "_id",
    foreignField: "client_id",
```

```

    as: "my_cmds"
  }},
  { $project: {
    recs: { $setDifference: [
      "$neighborhood_plats", // plats des voisins
      "$my_plats"           // plats déjà commandés
    ]}
  }}
}]

```

Explication : Cette requête implémente un algorithme de filtrage collaboratif basé sur la localisation géographique. Elle identifie les clients de la même ville, collecte l'ensemble de leurs plats commandés (neighborhood_plats via \$addToSet), puis utilise l'opérateur \$setDifference pour soustraire les plats déjà connus du client cible. Le résultat est une liste de plats « nouveaux » recommandés.

Opérateurs clés : \$match, \$lookup, \$group, \$addToSet, \$setDifference, \$project

The screenshot shows a web application interface with a sidebar on the left containing navigation items: Q4: Paiements, Q5: Tofu & Poisson, Q6: Dashboard Facet, Q7: GeoNear, Q8: Livreurs, Q9: Régimes, and Q10: Recommendations (highlighted in green). The main content area displays 'Question 10: Recommendations IA' with the subtitle 'Plats suggérés basés sur les voisins (même ville)'. Below this, a MongoDB pipeline is shown in a dark-themed code block:

```

PIPELINE MONGODB
db.clients.aggregate([
  { $match: { _id: ousmane_id } },
  { $lookup: { from: 'commandes', localField: '_id', foreignField: 'client_id', as: 'my_cmds' } },
  { $project: { recs: { $setDifference: ['$neighborhood_plats', '$my_plats'] } } }
])

```

Below the pipeline, a table displays the results:

Recommandation	Prix
Dibi Mouton	4500 FCFA
Mafé Tofu	2800 FCFA

Figure 15 – Q10 : Recommendations pour Ousmane Sarr (Dibi Mouton, Mafé Tofu)

4. Interface Web – Captures d'Écran

4.1 Page d'Accueil – Vue d'Ensemble

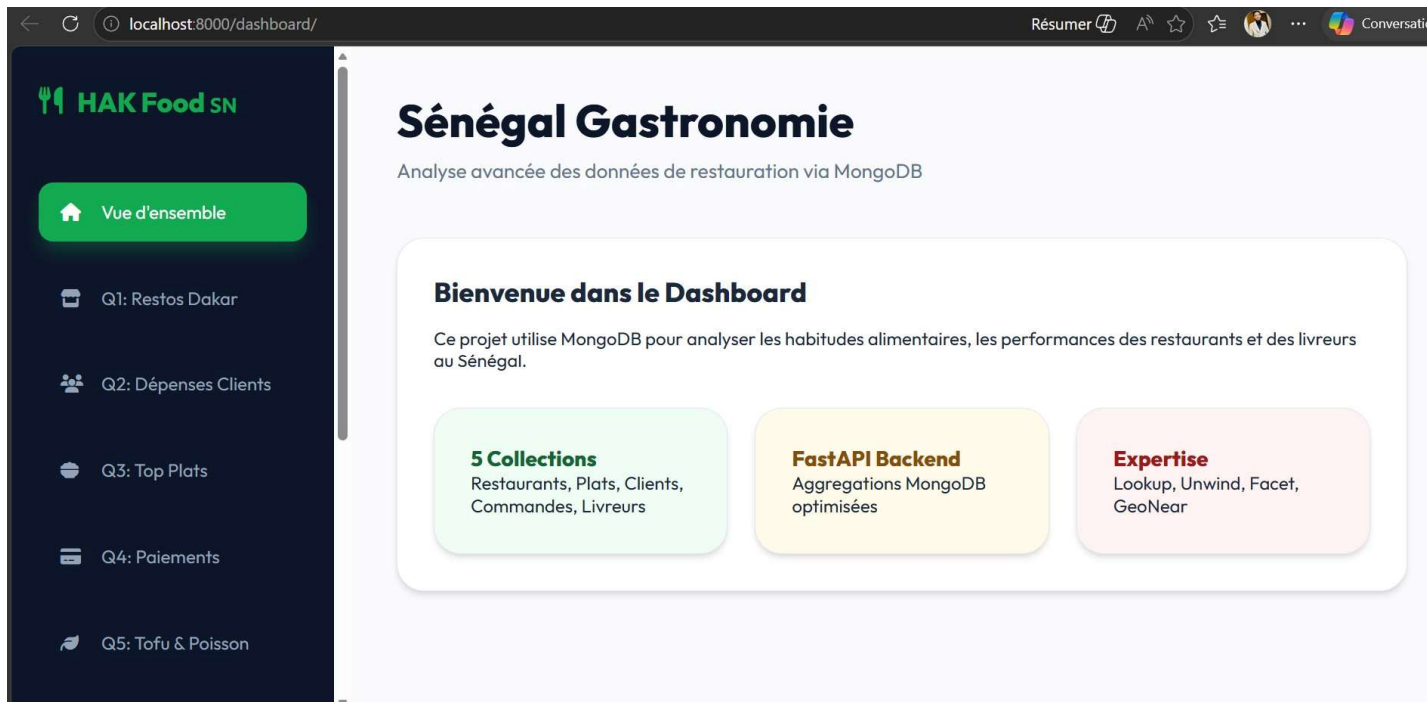


Figure 16 – Page d'accueil du dashboard HAK Food SN (localhost:8000/dashboard)

Le dashboard présente une navigation latérale permettant d'accéder rapidement aux 10 analyses. La page d'accueil met en avant les trois piliers du projet : 5 collections MongoDB, le backend FastAPI et les opérateurs d'expertise utilisés.

4.2 Dépôt GitHub

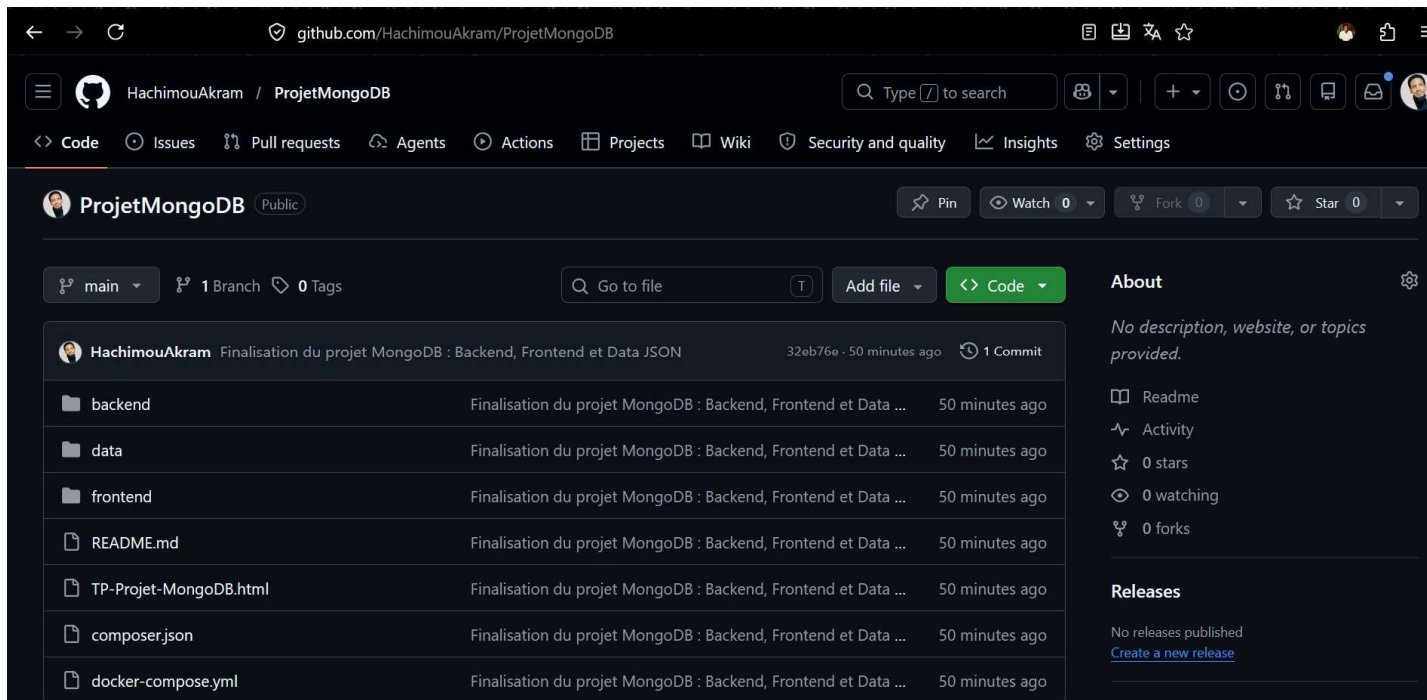


Figure 17 – Dépôt GitHub public (github.com/HachimouAkram/ProjetMongoDB)



Figure 18 – README.md du projet sur GitHub

5. Difficultés Rencontrées et Solutions Apportées

La réalisation de ce projet a présenté plusieurs défis techniques. Le tableau ci-dessous synthétise les principales difficultés rencontrées et les solutions implémentées :

Difficulté rencontrée	Solution apportée
Gestion des index géo-spatiaux (\$geoNear)	Création manuelle d'un index 2dsphere dans le script seed_db.py au démarrage de l'application, garantissant la disponibilité des index avant toute requête géospatiale.
Sérialisation des ObjectId MongoDB vers JSON	Implémentation d'une fonction utilitaire serialize_mongo() qui parcourt récursivement les documents et convertit les types BSON (ObjectId, datetime) en types JSON standard.
Orchestration Docker et ordre de démarrage	Configuration du depends_on avec condition: service_healthy dans docker-compose.yml et ajout d'un healthcheck sur le conteneur MongoDB pour garantir que la base est prête avant le lancement du backend FastAPI.
Requête Q5 sans résultat dans les données de test	La logique du pipeline (\$unwind → \$group avec \$addToSet → \$match avec \$all) est correcte. L'absence de résultat reflète fidèlement les données : aucun client n'a commandé à la fois du tofu ET du poisson dans le jeu de données fourni.
Requête Q10 (Recommandations)	Utilisation de \$setDifference entre les plats commandés par les voisins de la même ville et les plats déjà commandés par le client cible, via un pipeline multi-étapes avec \$lookup imbriqués.

Conclusion

Ce projet MongoDB – Alimentation & Gastronomie Sénégal a permis de démontrer que MongoDB, au-delà d'un simple stockage NoSQL, constitue un véritable moteur d'analyse capable de remplacer des traitements complexes côté applicatif par des pipelines d'agrégation optimisés.

Les dix requêtes implémentées couvrent l'ensemble des opérateurs avancés du cours : jointures multi-collections (\$lookup), déstructuration de tableaux (\$unwind), agrégations parallèles (\$facet), requêtes géospatiales (\$geoNear) et opérations ensemblistes (\$setDifference, \$addToSet). Chaque requête est exposée via une API REST FastAPI et visualisée dans un dashboard interactif.

L'architecture dockerisée garantit une reproductibilité complète du projet en une seule commande (docker-compose up --build), et le dépôt GitHub public permet une évaluation et une collaboration facilitées.

Ce travail renforce ma conviction que la maîtrise des pipelines d'agrégation MongoDB est une compétence différenciante pour tout développeur Full-Stack travaillant avec des données non relationnelles à grande échelle.